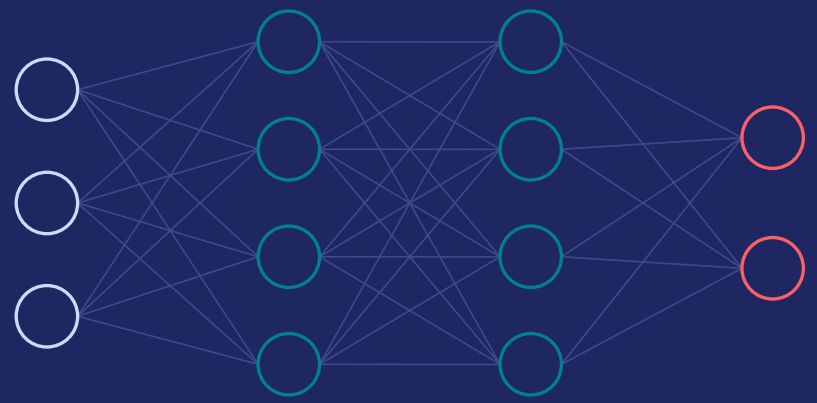
기계학습특론



CHAPTER 6-1

다층 퍼셉트론의 구조와 개념

퍼셉트론 → XOR → 다층 퍼셉트론(MLP) → 순전파 · 활성화 함수 · 최신 MLP 구조

성명 : 최승호
이메일 : jcn99250@naver.com

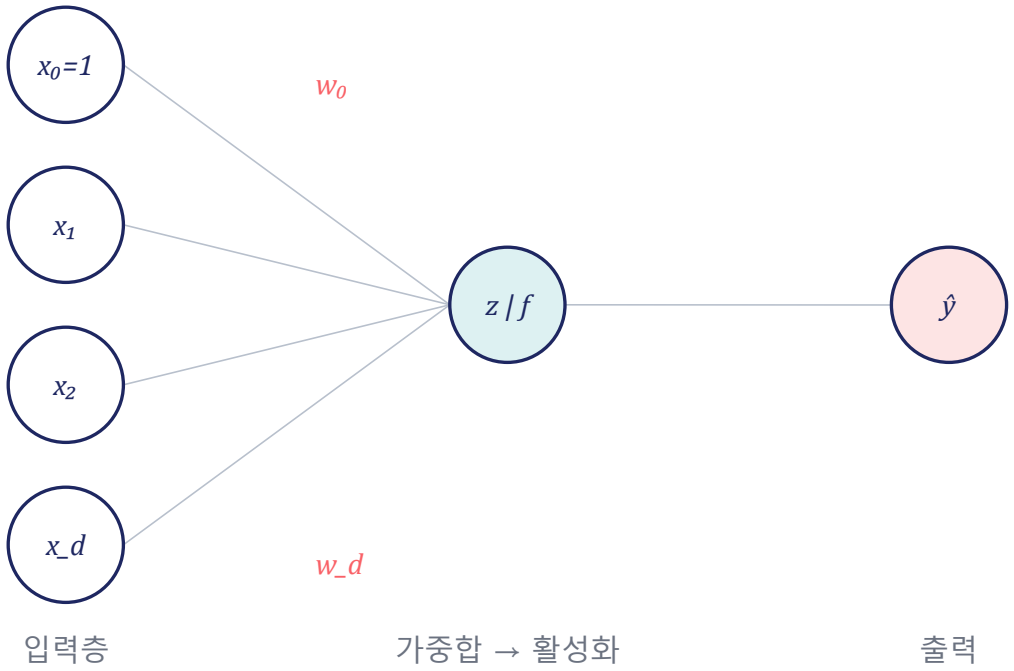
2026-2학기 · PyTorch 2.x 기준 개정판

학습 목표

1. 퍼셉트론의 구조(가중합 + 활성화 함수)를 복습하고 논리 게이트로 구현한다
2. XOR 문제가 단층 퍼셉트론으로 풀리지 않는 이유를 기하학적으로 이해한다
3. 다층 퍼셉트론(MLP)의 구조를 행렬 표기로 정의하고 순전파를 계산한다
4. 비선형 활성화 함수의 역할과 현대 활성화 함수(ReLU·GELU·SiLU)를 비교한다
5. MNIST 분류 MLP의 입력층·은닉층·출력층(소프트맥스) 동작을 이해한다
6. 보편 근사 정리, 초기화, 잔차 블록과 최신 동향(FFN/MoE·MLP-Mixer·KAN)을 살펴본다

퍼셉트론 복습

특징 벡터의 가중합을 활성화 함수에 통과시켜 출력을 계산하는 가장 단순한 신경 모델



$$\hat{y}_i = f(z_i) = f\left(\sum_{j=0}^{d} w_j x_{ij}\right) = f(\mathbf{w}^\top \mathbf{x}_i), \quad x_{i0} = 1$$

- **구성 요소**
  - 특징 벡터 $x_i = [x_{i1}, \ldots, x_{i_d}]^T$, 바이어스 입력 $x_{i0} = 1$ (w_0가 바이어스 역할)
  - 가중합 $z_i = w_0 + w_1 x_{i1} + \cdots + w_d x_{i_d} = w^T x_i$
  - 활성화 함수 f : 계단 함수(원조 퍼셉트론), 시그모이드, ReLU 등
- **학습 데이터 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ 로부터 가중치 w를 학습**
  - 출력 $\hat{y}_i$와 정답 y_i의 차이(손실)를 줄이는 방향으로 w를 갱신 → 6-2 «학습» 강의

퍼셉트론 학습 규칙 (Rosenblatt 1958)

오분류 샘플마다 $w \leftarrow w + \eta \cdot (y_i - \hat{y}_i) \cdot x_i$. 선형 분리 가능하면 유한 번에 수렴 (수렴 정리)

한계

결정 경계가 초평면 하나 → 선형 분리 불가능한 문제(XOR)는 영원히 못 푼다

현대적 관점

퍼셉트론 = 활성화 함수가 계단 함수인 «노드» 하나. 신경망은 이런 노드를 층으로 쌓은 것

퍼셉트론으로 논리 게이트 만들기

AND · OR · NAND는 가중치와 바이어스만 바꾸어 하나의 퍼셉트론으로 표현된다

$$z = x_1 + x_2 - 1.5 \text{ (AND)}, \qquad z = x_1 + x_2 - 0.5 \text{ (OR)}, \qquad z = -2x_1 - 2x_2 + 3 \text{ (NAND)}$$

x_1	x_2	AND	OR	NAND	XOR
0	0	0	0	1	0
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

실습 1 §1: w_nand=[-2,-2], b=3 / w_or=[2,2], b=-1 / w_and=[1,1], b=-1 로 직접 확인 (위 식의 AND $z = x_1+x_2-1.5$, OR $z = x_1+x_2-0.5$ 와 같은 게이트 — 같은 게이트를 만드는 해는 유일하지 않다)

결정 경계 (Decision boundary)

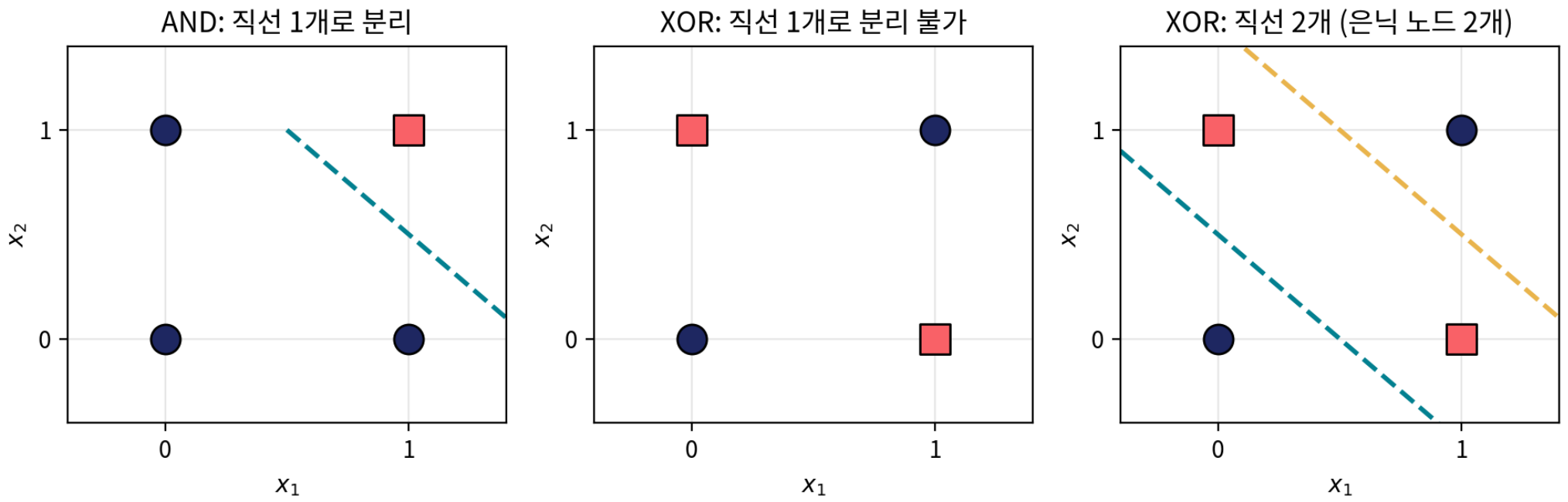
z = 0 인 직선이 입력 평면을 두 영역으로 나눈다. f(z) = 1[z > 0] 이면 직선 위쪽은 1, 아래쪽은 0.

선형 분리 가능 (Linearly separable)

AND · OR · NAND 는 한 직선으로 0과 1을 나눌 수 있다. XOR 은 어떨까? → 다음 슬라이드

XOR 게이트 (1/3) — 단층 퍼셉트론의 한계

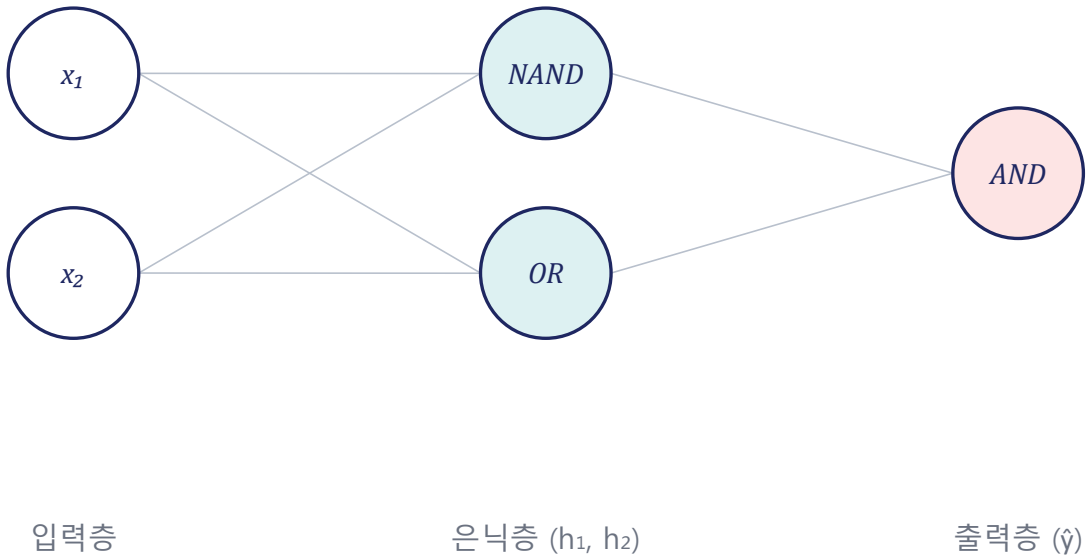
Minsky & Papert (1969)「Perceptrons」: 단층 퍼셉트론은 XOR을 표현할 수 없다



- XOR = 두 입력이 서로 다를 때 1, 같을 때 0 → (0,1), (1,0)이 1; (0,0), (1,1)이 0
- 대각선으로 마주 보는 두 점을 한 직선(초평면 하나)으로 분리하는 것은 불가능
- 해결책: 직선을 두 개 이상 사용 → 은닉층(hidden layer)에 퍼셉트론 여러 개를 두고 그 출력을 다시 결합

XOR 게이트 (2/3) — NAND · OR 의 출력을 AND 로 결합

은닉층 2개 노드(NAND, OR)와 출력층 1개 노드(AND) = 가장 작은 다층 퍼셉트론

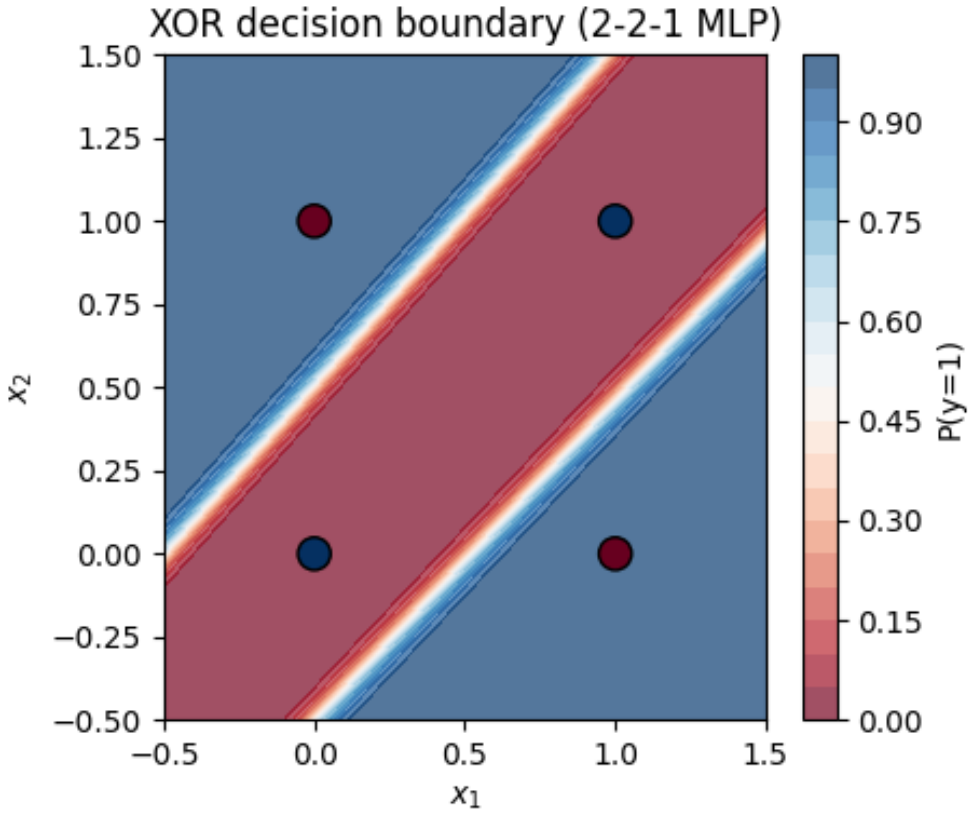


x_1	x_2	h_1 = NAND	h_2 = OR	$\hat{y}$ = AND(h_1,h_2)
0	0	1	0	0
0	1	1	1	1
1	0	1	1	1
1	1	0	1	0

- 은닉층은 입력을 새로운 좌표 (h_1, h_2)로 변환 → 변환된 공간에서는 XOR이 선형 분리 가능
- 즉, 은닉층 = 특징 변환(feature transformation), 출력층 = 변환된 특징의 선형 분류
- 이 관점이 딥러닝 전체를 관통한다: 층을 쌓을수록 더 복잡한 특징 변환을 학습

XOR 게이트 (3/3) — 학습으로 얻은 비선형 결정 경계

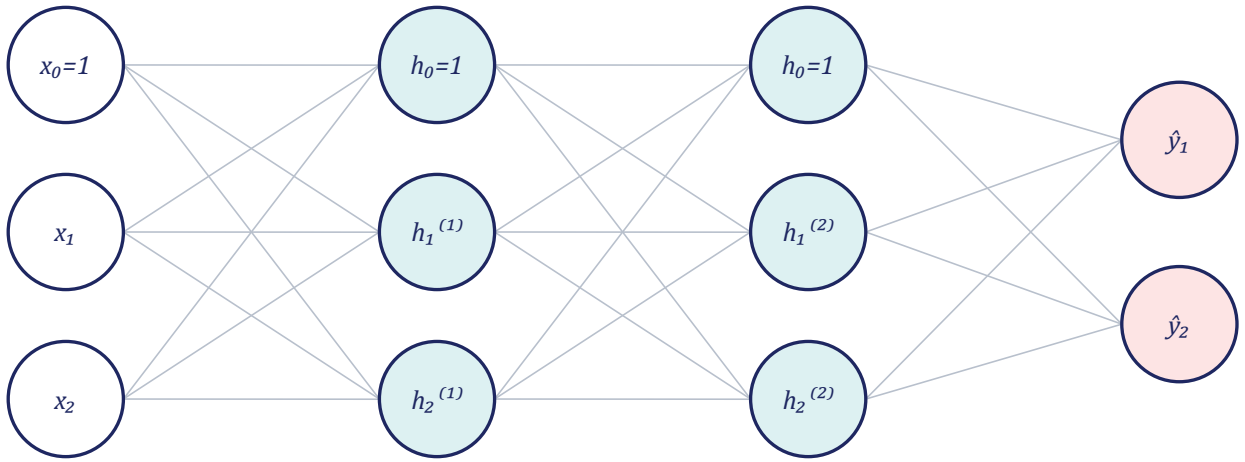
가중치를 손으로 정하지 않고 데이터로부터 학습해도 같은 해를 찾는다 (실습 1 §2)



- **2-2-1 MLP + 시그모이드, BCE 손실, 경사 하강법으로 학습**
  - 은닉 노드 2개가 각각 하나의 직선(결정 경계)을 만들고, 출력 노드가 두 직선을 결합
  - 학습 결과 예측 확률 ≈ [0.001, 0.999, 0.999, 0.001] → XOR 정답과 일치
- **직접 실험해 보기**
  - TensorFlow Playground (playground.tensorflow.org): 은닉층 0개 → 1개(노드 2~3개)로 늘리며 XOR 데이터가 언제 분리되는지 관찰
  - 실습 노트북: 시드·학습률을 바꾸면 실패(국소 최적)하는 경우도 있음 → 초기화의 중요성 (슬라이드 21)

다층 퍼셉트론 (Multi-Layer Perceptron, MLP)

입력층과 출력층 사이에 하나 이상의 은닉층이 있는 완전연결 순전파 신경망



Fully-Connected (완전연결)

인접한 두 층의 모든 노드 쌍이 가중치로 연결된다. k층 노드 수 n_k, k+1층 노드 수 n_{k+1} 이면 가중치 행렬은 n_{k+1} × n_k.

Feed-Forward (순전파)

정보가 입력 → 은닉 → 출력 방향으로만 흐른다 (순환 없음). 각 층은 이전 층의 출력만을 입력으로 받는다.

용어

층의 개수(깊이, depth)와 층당 노드 수(폭, width)가 구조를 결정. 은닉층이 많으면 «깊은(deep)» 신경망.

다층 퍼셉트론 요약 (1/2) — 행렬 표기

층 하나 = 선형 변환(가중치 행렬 + 바이어스) 뒤에 원소별 활성화 함수

$$\mathbf{z}^{(k)} = \mathbf{W}^{(k-1)}\mathbf{h}^{(k-1)} + \mathbf{b}^{(k-1)}, \qquad \mathbf{h}^{(k)} = f_k(\mathbf{z}^{(k)}), \qquad \mathbf{h}^{(0)} = \mathbf{x}$$

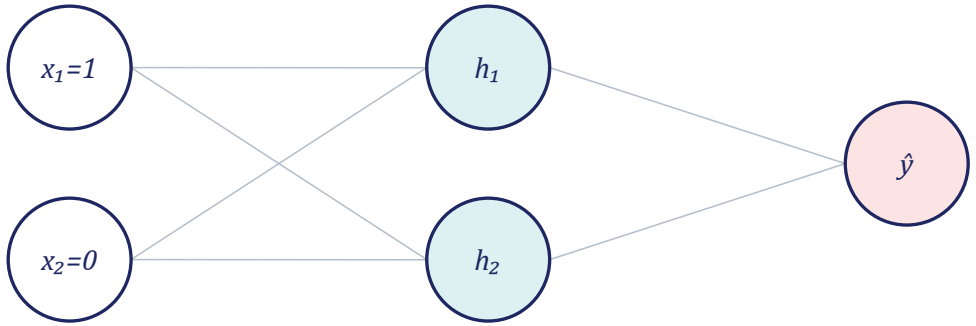
$$\hat{\mathbf{y}} = f_2(\mathbf{W}^{(1)} f_1(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)})$$

- **표기**
  - $W^{(k)}$: k층 → k+1층 가중치 행렬, $b^{(k)}$: 바이어스 벡터 (그림의 w_{10}, w_{20} 열을 분리한 것)
  - $z^{(k)}$: k층 가중합(pre-activation), $h^{(k)}$: k층 출력(activation), $h^{(0)} = x$
  - f_k : k층 활성화 함수 — 은닉층은 ReLU 계열, 출력층은 문제에 따라 항등/시그모이드/소프트맥스
- **2층 MLP (은닉층 1개)는 함수 합성 $\hat{y} = f_2(W^{(1)} f_1(W^{(0)}x + b^{(0)}) + b^{(1)})$**
  - 은닉층 개수가 늘어나도 같은 규칙을 반복 적용 — 코드로는 for 문 한 개

기호	크기 (2-2-2 예)	PyTorch
x	2×1	입력 텐서 (B, 2)
$W^{(0)}$, $b^{(0)}$	2×2, 2×1	nn.Linear(2, 2)
$h^{(1)}$	2×1	활성화 출력
$W^{(1)}$, $b^{(1)}$	2×2, 2×1	nn.Linear(2, 2)
$\hat{y}$	2×1	로짓 / 확률

다층 퍼셉트론 요약 (2/2) — 순전파 계산 예

2-2-1 구조에서 숫자를 넣어 손으로 따라가 보기 (실습 1 §1과 동일한 가중치 배치)




```
# W0 = [[-2, -2],     b0 = [ 3, -1]     (행 = NAND, OR 노드)
#       [ 2,  2]]
# W1 = [[ 1,  1]],   b1 = [-1]          (출력노드 AND)
x  = [1, 0]
z1 = W0 @ x + b0 = [(-2)·1 + (-2)·0 + 3,  2·1 + 2·0 - 1] = [1, 1]
h1 = step(z1)     = [1, 1]          # (NAND, OR) 출력
z2 = W1 @ h1 + b1 = 1 + 1 - 1 = 1
ŷ  = step(z2)     = 1               # XOR(1, 0) = 1  ✓
```

- **핵심 관찰**
  - 순전파는 «행렬곱 → 바이어스 덧셈 → 활성화» 의 반복. GPU가 잘하는 연산(행렬곱)으로만 구성된다
  - 미니배치 B개를 한 번에 처리하려면 x를 (B × d) 행렬로 쌓으면 된다: $Z = X W^{T} + b$ (PyTorch nn.Linear 의 규약)
  - 계단 함수는 미분 불가 → 학습하려면 시그모이드·ReLU 같은 미분 가능한 활성화 함수가 필요 (6-2 강의)

비선형 활성화 함수가 없으면 층을 쌓아도 소용없다

선형 변환의 합성은 다시 하나의 선형 변환 — 은닉층의 의미는 비선형성에서 나온다

$$\mathbf{W}^{(1)}(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)} = (\mathbf{W}^{(1)}\mathbf{W}^{(0)})\mathbf{x} + (\mathbf{W}^{(1)}\mathbf{b}^{(0)} + \mathbf{b}^{(1)}) = \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}}$$

선형 (f = 항등)

- $W^{(1)}W^{(0)}$ 는 또 하나의 행렬
- 아무리 깊어도 직선(초평면) 하나
- XOR 해결 불가

비선형 (ReLU, 시그모이드…)

- 층마다 공간을 «접고 구부린다»
- 은닉 노드 하나 = 직선 하나, 결합하면 임의의 영역
- XOR 해결 가능

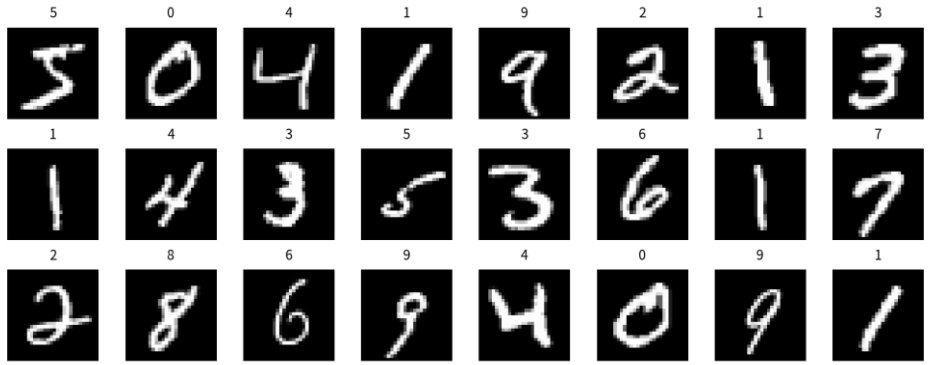
실습 확인

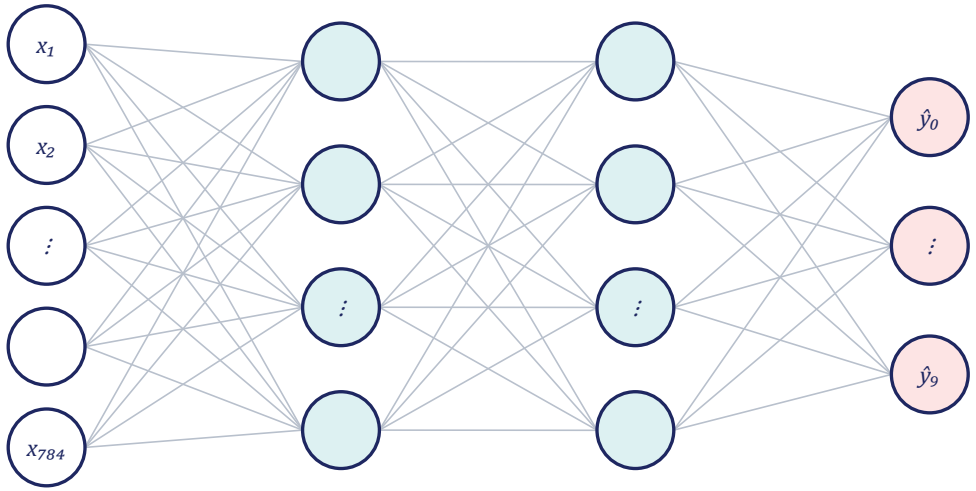
- nn.Sequential 에서 nn.Sigmoid() 를 제거하고 학습
- 손실이 0.69(ln 2) 근처에서 정체 → 무작위 추측 수준
- 실습 1 «생각해 볼 문제 1»

정리: MLP = (선형 → 비선형) 블록의 반복. 표현력은 비선형성과 깊이·폭에서, 학습 가능성은 미분 가능성에서 나온다.

예시: MNIST 손글씨 분류를 위한 MLP

28×28 흑백 이미지 → 784차원 벡터 → 은닉층 2개 → 10개 클래스 확률





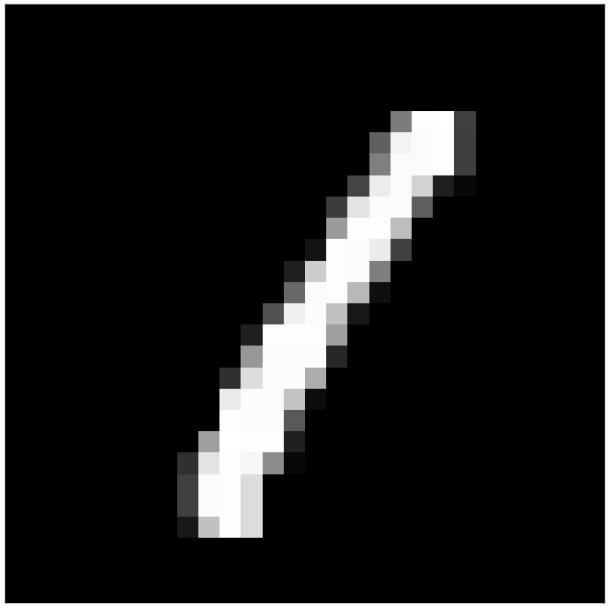
입력층 784 은닉층 1 (600) 은닉층 2 (500) 출력층 10

- **MNIST (Modified NIST) 데이터셋**
  - 훈련 60,000장 · 테스트 10,000장, 0~9 손글씨 숫자, 28×28 픽셀 (0~255 회색조)
  - 기계학습·딥러닝 모델의 «Hello, World» — 단순하지만 입력·은닉·출력층의 역할을 모두 보여 준다
- **구조 784 → 600 → 500 → 10 (실습 2와 동일)**
  - 파라미터 수 = 784·600+600 + 600·500+500 + 500·10+10 = 776,510
  - 입력층: 이미지를 벡터로 펼침(flatten) — 픽셀의 2차원 위치 정보는 버려진다 (CNN이 해결, 10주차)
  - 출력층: 소프트맥스로 10개 클래스 확률 → 가장 큰 확률의 클래스로 분류

입력층 — 이미지를 벡터로, 레이블을 원-핫 벡터로

학습 데이터 $\mathcal{D} = \{(x_i, y_i)\}$ 의 $x_i \in \mathbb{R}^{784}$, $y_i \in \{0,\ldots,9\}$

28×28 = 784 픽셀 → 벡터 x ∈ R^784

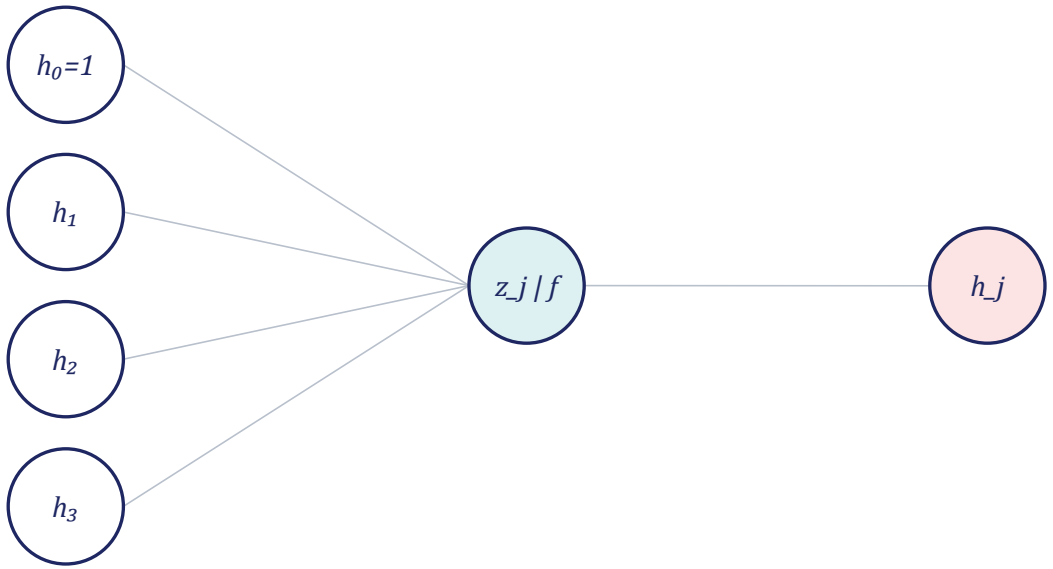


```
# 16 픽셀(4×4) 축소 예: 숫자 "1"
X = [[  0,   0, 255,   0],
     [  0, 255, 255,   0],
     [  0,   0, 255,   0],
     [  0,   0, 255,   0]]
x = flatten(X) / 255            # (16,)  → 실제로는 (784,)
x = (x - 0.1307) / 0.3081       # 표준화 (MNIST 평균·표준편차)
y = 1  →  one_hot(y) = [0,1,0,0,0,0,0,0,0,0]
```

- **입력층 노드 수 = 특징 벡터 차원 (바이어스 노드 $x_0=1$ 은 별도로 표시)**
  - 입력층은 계산하지 않고 값을 «전달»만 한다. 관측값 (x_i, y_i)의 x_i 성분이 그대로 노드 값
  - 정규화·표준화: 픽셀을 0~1 또는 평균 0·분산 1로 맞추면 가중합의 크기가 안정되어 학습이 빨라진다
- **원-핫 인코딩 (One-hot encoding)**
  - 클래스 개수만큼의 성분 중 정답 위치만 1. 소프트맥스 출력 $\hat{y}$와 같은 크기 → 손실(CCE) 계산 가능
  - PyTorch nn.CrossEntropyLoss 는 정수 레이블을 받아 내부에서 원-핫과 동일하게 처리

은닉층 — 노드 하나의 동작

입력: 이전 층 출력 $h^{(k-1)}$ 와 가중치 $W^{(k-1)}$ · 출력: 가중합 z 에 활성화 함수를 적용한 $h^{(k)}$



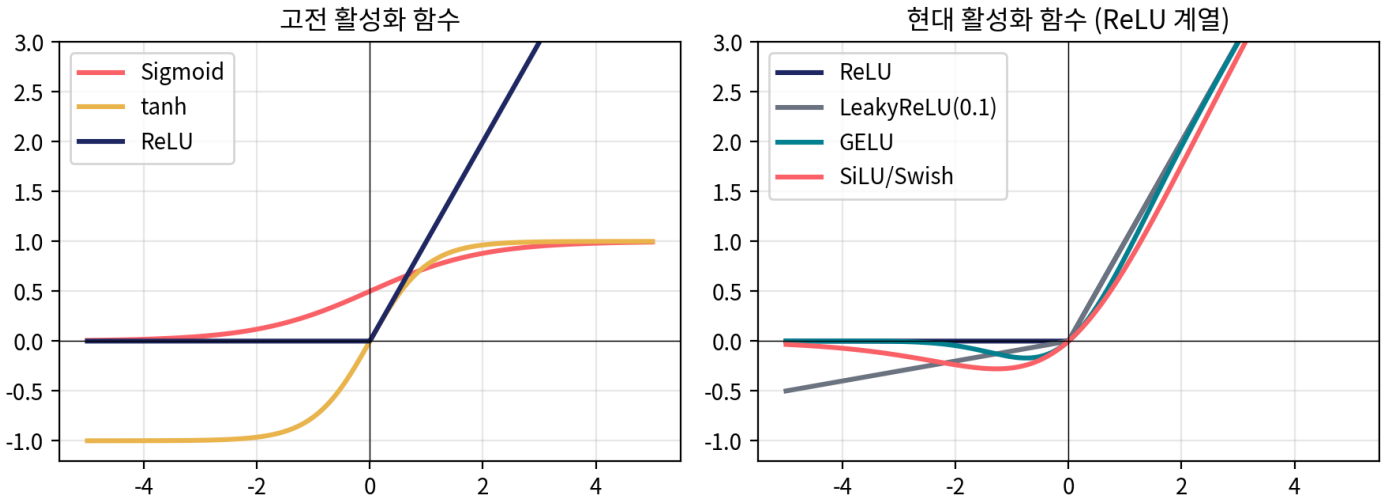
이전 층 출력 j번째 은닉 노드 출력

$$\mathrm{ReLU}(z) = \max(0, z) \qquad \mathrm{GELU}(z) = z\,\Phi(z) \qquad \mathrm{SiLU}(z) = z\,\sigma(z)$$

- 은닉층은 여러 개 존재할 수 있으며 각 노드는 앞 층의 모든 노드와 연결 (완전연결)
- 노드 j의 가중합: $z_j = \Sigma_i\, w_{ji} h_i + b_j$ — 앞 층 노드 수만큼의 곱셈·덧셈
- **활성화 함수는 주로 ReLU 계열**
  - ReLU(z) = max(0, z): 0보다 크면 그대로 통과, 아니면 0 → 계산이 싸고 기울기가 0 또는 1
  - GELU / SiLU: 0 근처를 부드럽게 만든 ReLU — 트랜스포머(BERT, GPT, LLaMA)의 기본 선택
  - 시그모이드·tanh는 은닉층에서 잘 쓰지 않는다 (포화 → 기울기 소실, 다음 슬라이드)
- 노드 수(폭)는 하이퍼파라미터: 넓을수록 표현력↑, 파라미터·과적합 위험↑

활성화 함수 비교 — 고전 vs 현대

2012년 이후 ReLU가 표준이 되었고, 2018년 이후 대형 모델은 GELU · SiLU(Swish) · SwiGLU를 채택

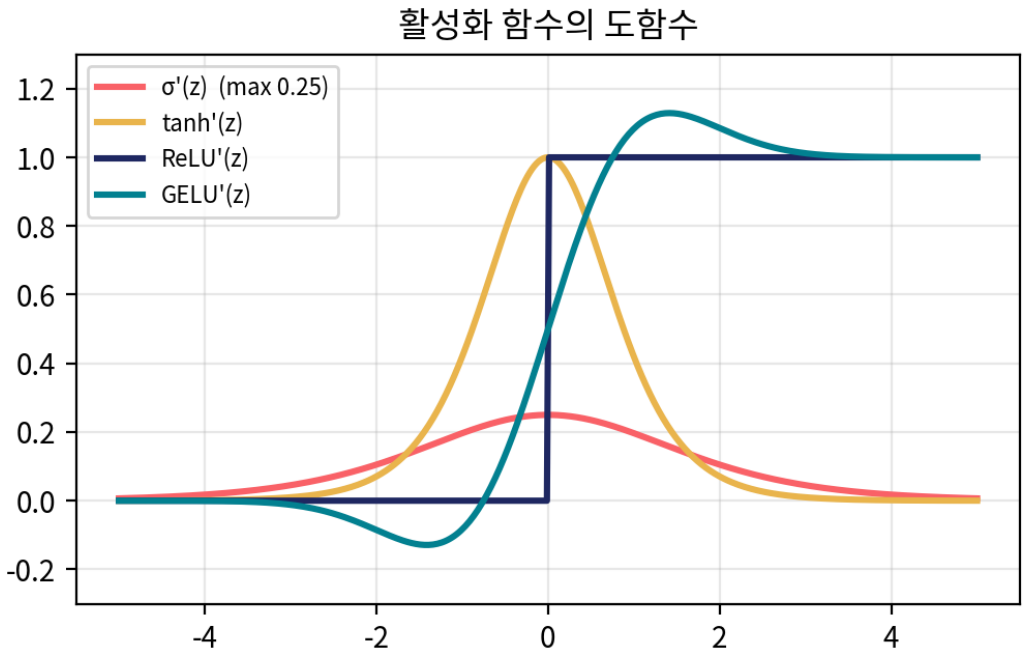


함수	식	특징 / 주 사용처
Sigmoid	$1/(1+e^{-z})$	출력 (0,1); 이진 분류 출력층
tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	출력 (–1,1); RNN 게이트
ReLU	max(0, z)	은닉층 기본값; dying ReLU 문제
LeakyReLU	max(αz, z)	음수 영역에 작은 기울기 α
GELU	$z\cdot\Phi(z)$	BERT, GPT-2/3, ViT
SiLU / Swish	$z\cdot\sigma(z)$	EfficientNet, LLaMA(SwiGLU)

- 시그모이드·tanh: |z|가 크면 기울기가 0에 가까워지는 «포화» → 깊은 망에서 학습 정체
- ReLU: 양수 영역 기울기 1 → 깊어도 기울기가 전달. 단, z<0 인 노드는 기울기 0 → 영원히 죽을 수 있음 (dying ReLU)
- GELU·SiLU: 0 근처가 매끄럽고 음수에서도 작은 기울기 → 실습 1의 XOR 실험에서 GELU가 10/10 성공 (ReLU 6/10)

활성화 함수의 도함수와 기울기 소실

역전파는 도함수를 층마다 곱한다 — σ'(z) ≤ 1/4 이면 10층만 지나도 $(1/4)^{10} \approx 10^{-6}$



$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)\,(1-\sigma(z)) \le \frac{1}{4}$$

- **기울기 소실 (vanishing gradient)**
  - 출력층에서 입력층으로 갈수록 기울기가 지수적으로 줄어 앞쪽 층이 학습되지 않음
  - 1990~2000년대 «깊은 신경망은 학습이 안 된다»의 주범 → ReLU(2011), 초기화(2010/2015), BatchNorm(2015), 잔차 연결(2015)로 해결
- ReLU'(z) ∈ {0, 1}: 활성 노드는 기울기를 그대로 통과 → 깊이가 수십~수백 층이어도 학습 가능
- GELU'(z): 음수 영역에서도 0이 아닌 값 → dying ReLU 완화

자세한 내용은 6-2 강의 «오차 역전파» 및 «기울기 소실·폭주» 슬라이드에서 수식과 함께 다룬다

출력층 — 소프트맥스 활성화 함수

다중 클래스 분류: C개의 로짓 z를 합이 1인 확률 벡터 ŷ로 변환

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}, \qquad \sum_k \hat{y}_k = 1$$

$$\mathbf{z} = [0.05,\ 0.11,\ 1.4,\ \ldots,\ 8.2]^\top \Rightarrow \hat{y}_{10} = \frac{e^{8.2}}{\sum_j e^{z_j}} \approx 0.64$$

k	z_k	e^{z_k}	ŷ_k
1	0.05	1.05	0.0002
2	0.11	1.12	0.0002
…	…	…	…
9	7.1	1212	0.21
10	8.2	3641	0.64

- **분류 규칙: argmax_k ŷ_k → 위 예에서는 10번째 클래스 (숫자 9)**
- 출력 노드 수 = 클래스 수 C. 이진 분류(C=2)는 시그모이드 출력 1개와 동치
- 출력층 활성화 함수는 문제 유형이 결정: 회귀 → 항등(선형), 이진 → 시그모이드, 다중 → 소프트맥스
- 구현 주의: PyTorch nn.CrossEntropyLoss 는 로짓 z를 입력받아 log-softmax를 내부에서 계산 → 모델 마지막에 Softmax를 두지 않는다 (수치 안정성, 이중 소프트맥스 방지)

원-핫 인코딩과 분류 결과 해석

소프트맥스 출력 $\hat{y}$ 와 원-핫 정답 y 를 나란히 놓으면 «얼마나 틀렸는지»가 바로 보인다

클래스	0	1	2	3	4	5	6	7	8	9
y (정답 = 1)	0	1	0	0	0	0	0	0	0	0
$\hat{y}$ (예시 1)	0.10	0.80	0.01	0.004	0.001	0.0004	0.0006	0.009	0.07	0.005
$\hat{y}$ (예시 2)	0.05	0.30	0.02	0.01	0.02	0.03	0.01	0.02	0.51	0.03

예시 1 → 정답

- argmax $\hat{y}$ = 1 = y
- 정답 확률 0.80 → 손실 $-\ln 0.80 \approx 0.22$
- 확신 있게 맞춤

예시 2 → 오답

- argmax $\hat{y}$ = 8 ≠ y
- 정답 확률 0.30 → 손실 $-\ln 0.30 \approx 1.20$
- 1과 8을 혼동 (혼동행렬에서 확인 가능)

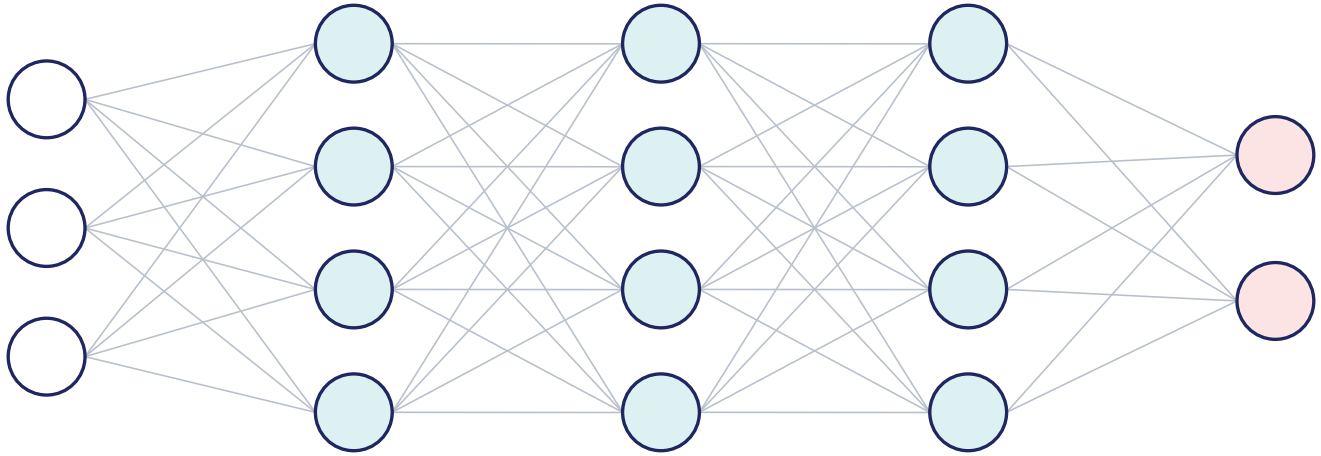
학습의 목표

- 정답 클래스의 확률 $\hat{y}_y$ 를 1에 가깝게
- = 교차 엔트로피 손실 $-\ln \hat{y}_y$ 최소화
- 6-2 강의에서 손실 함수로 정식화

혼동행렬(confusion matrix)은 «어떤 클래스끼리 혼동하는가»를 보여 주는 표 — 실습 2 §5 에서 4↔9, 3↔5, 7↔2 확인

다수의 은닉층이 존재하는 다층 퍼셉트론

깊이(depth)와 폭(width)은 표현력·파라미터 수·학습 난이도를 함께 결정한다



구조 (MNIST)	파라미터 수	비고
784-10 (로지스틱 회귀)	7,850	선형 분류기, ~92%
784-600-500-10	776,510	실습 2 기본, ~98.5%
784-1024-1024-10	1,863,690	폭 확장
784-256×8-10	664,074	깊이 확장 (8층)

- k−1층의 모든 노드와 k층의 모든 노드가 연결 → 층 사이 파라미터 수 = n_{k−1}·n_k + n_k
- 깊이가 주는 이점: 같은 파라미터로 더 복잡한 함수(계층적 특징) 표현 — 낮은 층은 단순 패턴, 높은 층은 조합
- 깊이가 주는 어려움: 기울기 소실·폭주, 최적화 난이도 → 초기화·정규화·잔차 연결이 필수 (슬라이드 21~22)

실습 2 «생각해 볼 문제 2·4»: 시그모이드 5층 vs BatchNorm/잔차 추가, 폭 1024 확장 실험

이론: 보편 근사 정리 (Universal Approximation Theorem)

은닉층 1개짜리 MLP도 충분히 넓으면 콤팩트 집합 위의 임의의 연속함수를 원하는 정밀도로 근사할 수 있다

$$\forall \varepsilon > 0 \;\; \exists N,\ \mathbf{W}, \mathbf{b}: \;\; \sup_{\mathbf{x} \in K} \left| g(\mathbf{x}) - \sum_{j=1}^{N} v_j \, f(\mathbf{w}_j^{\top} \mathbf{x} + b_j) \right| < \varepsilon$$

고전 결과

- Cybenko (1989): 시그모이드
- Hornik (1991): 임의의 비상수·유계·연속 활성화
- Leshno et al. (1993): 비다항식 활성화면 충분 (ReLU 포함)

정리가 말하지 않는 것

- 필요한 폭 N이 얼마나 큰가 → 최악의 경우 차원에 지수적
- 그 가중치를 경사 하강법이 «찾을 수 있는가» (최적화)
- 새로운 데이터에 «일반화»되는가

깊이의 이점 (현대 결과)

- 깊이 분리 정리 (Telgarsky 2016; Eldan & Shamir 2016): 어떤 함수는 깊은 망은 작게, 얕은 망은 지수적으로 넓어야 표현
- 폭 제한 UAT: 폭 ≥ d+1 이면 깊이만으로도 보편 근사 (Lu et al. 2017 은 폭 d+4, 최소 폭 d+1 확정은 Park et al. 2021)

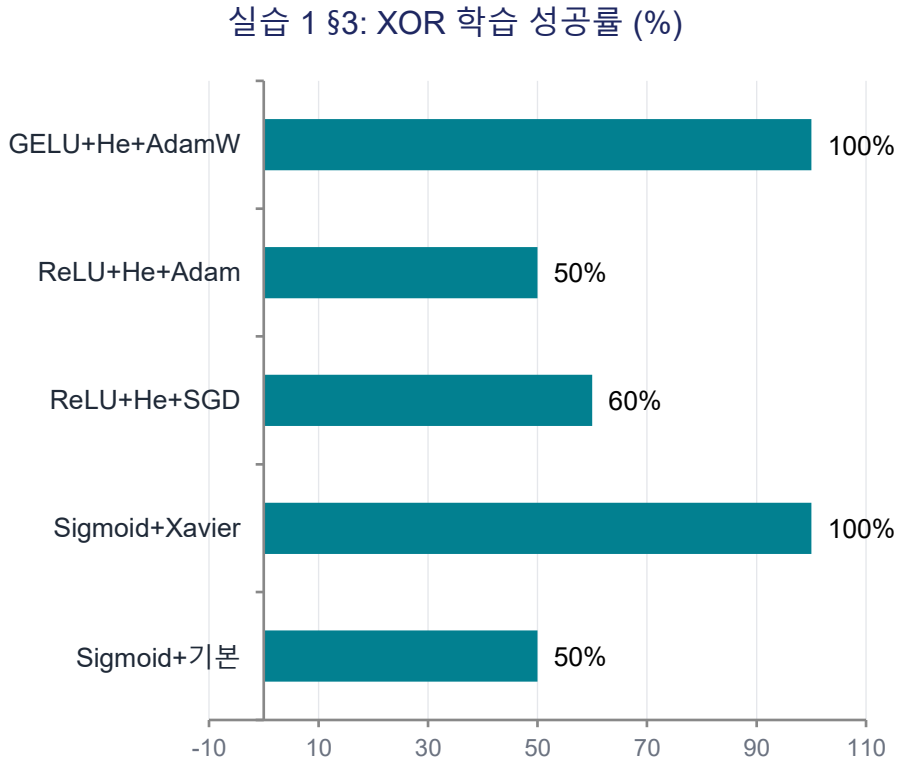
결론: «표현할 수 있다»(UAT)와 «학습해서 잘 맞춘다»(최적화 + 일반화)는 다른 문제 — 6-2 강의가 후자를 다룬다

현대 기법 ①: 가중치 초기화 (Xavier · He)

초기 가중치의 분산을 층 크기에 맞추면 순전파 신호와 역전파 기울기의 크기가 층을 지나도 유지된다

$$w \sim \mathcal{N}\left(0,\ \frac{2}{n_{in}}\right)\ \text{(He)}, \qquad w \sim \mathcal{U}\left(\pm\sqrt{\frac{6}{n_{in}+n_{out}}}\right)\ \text{(Xavier)}$$

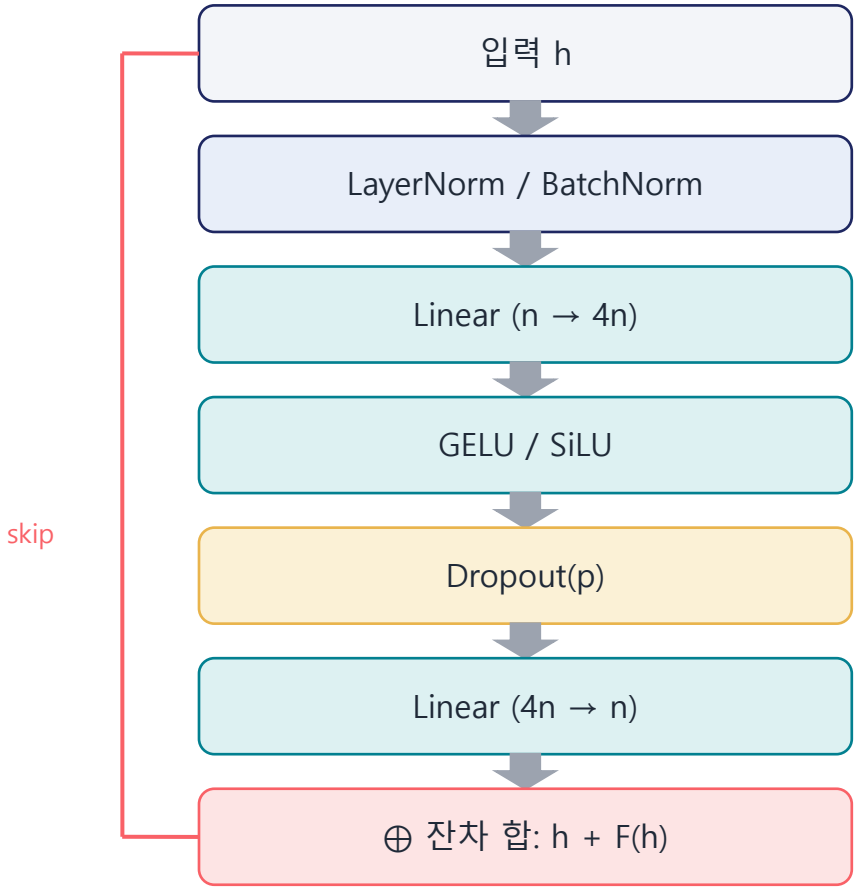
- Xavier/Glorot (2010): 시그모이드·tanh 용, 입력·출력 노드 수의 평균으로 분산 조절
- He/Kaiming (2015): ReLU 는 절반을 0으로 만들므로 분산을 2배 → Var(w) = 2/n_in
- PyTorch: nn.init.kaiming_normal_(W, nonlinearity='relu'), nn.init.xavier_uniform_(W). nn.Linear 기본값은 Kaiming-uniform(a=√5) 변형
- 0으로 초기화하면 모든 노드가 같은 값·같은 기울기를 가져 «대칭»이 깨지지 않음 → 절대 금지



같은 2-2-1 구조에서도 초기화·활성화·최적화기에 따라 성공률이 50% ↔ 100% 로 갈린다 (1000 epoch, lr 고정)

현대 기법 ②: 잔차 연결 · 정규화 · 드롭아웃을 갖춘 MLP 블록

«깊게 쌓아도 학습되는» 블록 — ResNet(2015)과 트랜스포머(2017)가 정착시킨 표준 설계



잔차 연결 (Residual, 2015)

- h_{k+1} = h_k + F(h_k)
- 기울기가 항등 경로로 직접 전달 → 100층 이상도 학습
- 블록이 «변화량»만 학습하면 됨

정규화 (BN 2015 / LN 2016)

- 각 층 입력의 분포를 평균 0·분산 1로 재조정
- 학습률을 크게 써도 안정, 초기화 민감도 ↓
- 트랜스포머는 Pre-LN 배치가 표준

드롭아웃 (2014)

- 학습 시 노드를 확률 p로 무작위 끔
- 특정 노드 의존을 막아 과적합 완화
- 추론 시에는 모든 노드 사용 (model.eval())

확장 비율 4×

- 중간 폭을 4배로 넓혔다 줄이는 «역병목 (inverted bottleneck)» 구조
- GPT 계열의 FFN 이 정확히 이 형태 (LLaMA 는 SwiGLU 변형)
- 파라미터의 약 2/3가 여기에 있다

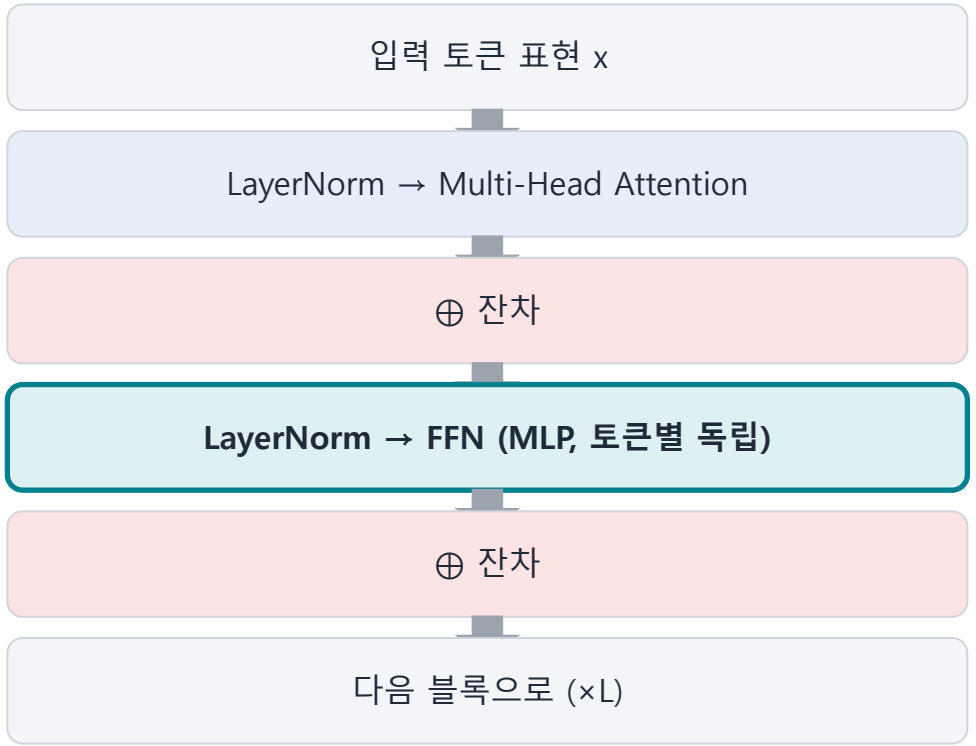
실습 2 §6 실험 D: GELU + BatchNorm + Dropout(0.2) + AdamW → 3 epoch 만에 검증 정확도 98.0%

최신 동향 ①: 트랜스포머 안의 MLP — FFN 과 SwiGLU

LLM 파라미터의 약 2/3는 어텐션이 아니라 토큰별로 독립 적용되는 «위치별 MLP»(FFN)에 있다

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2\,(\mathrm{SiLU}(\mathbf{W}_1\mathbf{x}) \odot \mathbf{W}_3\mathbf{x})$$

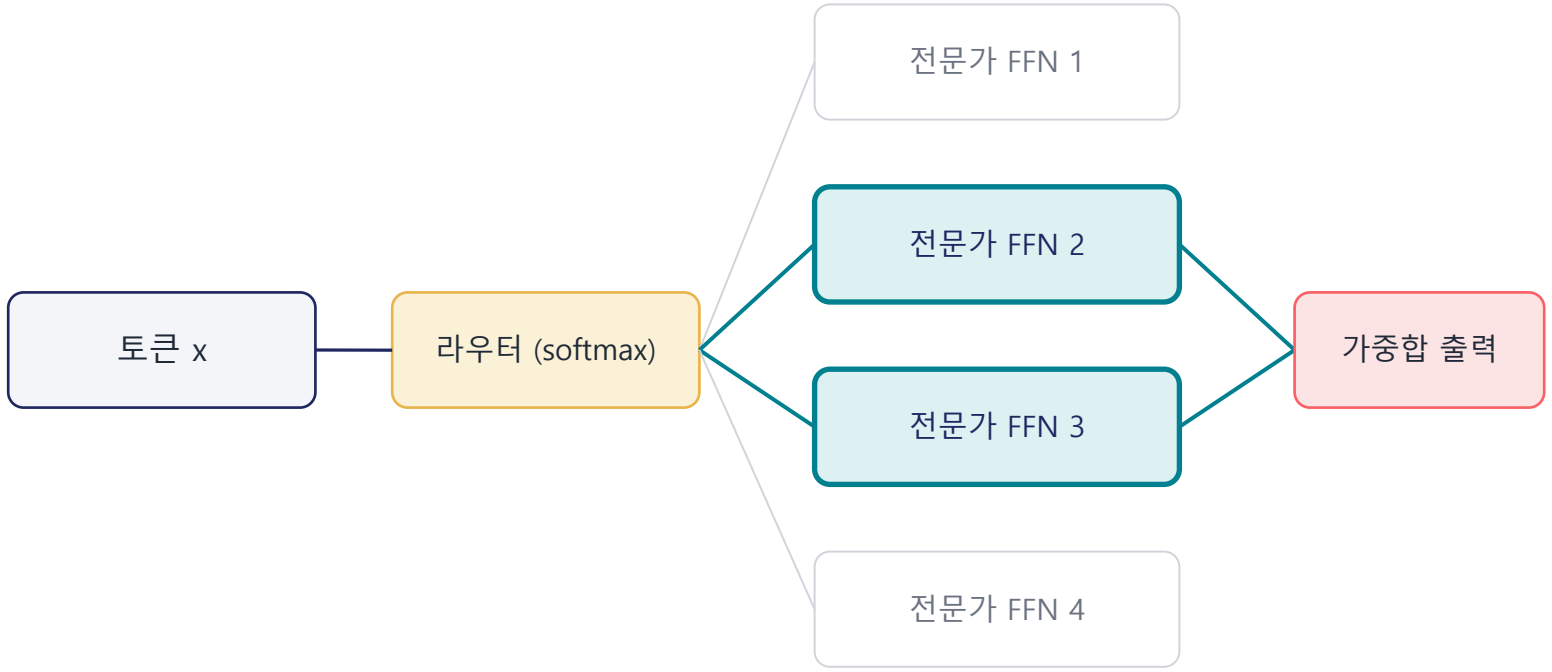
- 원래 FFN (Vaswani et al. 2017): $\mathrm{FFN}(x) = W_2\,\mathrm{ReLU}(W_1 x + b_1) + b_2$, 은닉 폭 = $4 \cdot d_model$
- GELU 채택 (BERT 2018, GPT-2 2019) → 매끄러운 활성화가 언어 모델에서 일관되게 유리
- GLU 변형 (Shazeer 2020): 두 선형 변환의 원소별 곱(게이트) — SwiGLU 는 PaLM(2022), LLaMA 1~3(2023~24), Qwen, Mistral 등의 표준
- 게이트가 있으면 은닉 폭을 $8/3 \cdot d$ 안팎으로 줄여 파라미터를 맞춘다 (행렬 3개; LLaMA-3 8B 는 3.5d)
- 해석: FFN 은 «키-값 메모리» — 첫 행렬의 행이 패턴을 감지(키), 둘째 행렬의 열이 출력 방향(값) (Geva et al. 2021)



모델	d_model	FFN 폭	활성화
GPT-2 (2019)	768~1600	4d	GELU
LLaMA-3 8B	4096	14336 (3.5d)	SwiGLU

최신 동향 ②: 전문가 혼합 (Mixture of Experts, MoE)

FFN을 여러 개(전문가) 두고 토큰마다 일부만 활성화 → 파라미터는 크게, 계산량은 작게



- **희소 활성화 (sparse activation)**
  - 총 파라미터 ≫ 토큰당 활성 파라미터. 예: Mixtral 8×7B — 총 47B, 활성 13B
  - Switch Transformer (2021), GLaM (2021), Mixtral (2023), DeepSeek-V2/V3 (2024), Llama 4 (2025) 등이 채택
- **훈련 이슈**
  - 부하 불균형(모든 토큰이 같은 전문가로) → 보조 손실 또는 손실 없는 균형 기법 (DeepSeek)
  - 라우팅은 이산 선택 → 미분 불가, top-k 게이트 값을 통해 우회 학습
- **본질은 여전히 «MLP 여러 개 + 가중합» — 6주차 내용의 직접 확장**

Top-k (k=2) 선택: 라우터 점수가 높은 전문가만 계산

최신 동향 ③: MLP-Mixer — 어텐션·합성곱 없이 MLP만으로 이미지 분류

Tolstikhin et al. (NeurIPS 2021): 패치 × 채널 행렬에 MLP를 두 방향으로 번갈아 적용



- **핵심 아이디어**
  - 합성곱(CNN)이나 자기-어텐션(ViT) 없이도 «두 방향 MLP»만으로 ImageNet에서 경쟁력 있는 정확도
  - Token-mixing = 모든 채널에 같은 가중치 공유 (합성곱과 유사한 귀납 편향), Channel-mixing = 1×1 합성곱과 동일
- **시사점**
  - 대규모 데이터·계산에서는 구조적 귀납 편향보다 «단순한 구조 + 스케일»이 중요할 수 있음
  - 후속: gMLP (2021), ResMLP (2021), ConvMixer (2022) — MLP 계열 비전 백본
  - 한계: 입력 해상도(패치 수)가 고정되어야 하며, 소규모 데이터에서는 CNN/ViT 에 뒤짐

최신 동향 ④: KAN (Kolmogorov–Arnold Networks)

Liu et al. (2024): 노드가 아니라 «엣지»에 학습 가능한 1차원 함수(스플라인)를 두는 MLP의 대안

$$\text{KAN}: \mathbf{h}_i^{(k+1)} = \sum_j \phi_{ij}\left(h_j^{(k)}\right) \quad \text{vs.} \quad \text{MLP}: \mathbf{h}^{(k+1)} = f(\mathbf{W}\mathbf{h}^{(k)})$$

	MLP	KAN
이론적 근거	보편 근사 정리	Kolmogorov–Arnold 표현 정리
학습 대상	선형 가중치 W (엣지), 고정 활성화 (노드)	엣지마다 학습 가능한 1D 함수 φ_ij (B-spline), 노드는 합만
장점	GPU 행렬곱에 최적, 대규모 검증됨	매개변수 효율·해석 가능성 (기호 회귀, 과학 발견)
단점	블랙박스, 스케일 의존	학습 느림, 대규모 과제에서 MLP 대비 우위 불확실

- 후속 연구: KAN 2.0 (2024), FastKAN/ChebyKAN 등 기저 함수 변형, «KAN or MLP: A Fairer Comparison» (Yu et al. 2024) — 기호 회귀 외에는 MLP 우세 보고
- 교훈: «어디에 비선형성을 두는가»는 여전히 열린 설계 문제. 6주차의 MLP 정의를 정확히 이해해야 이런 변형을 읽고 평가할 수 있다

실습 안내: PyTorch로 XOR 다층 퍼셉트론 (6주차_MLP_실습1)

TensorFlow 버전을 PyTorch 2.x 로 개정 — 손 계산 역전파와 autograd 검증, nn.Module 구현, 초기화·활성화 비교 실험

```
import torch, torch.nn as nn
X = torch.tensor([[0.,0.],[0.,1.],[1.,0.],[1.,1.]])
Y = torch.tensor([[0.],[1.],[1.],[0.]])

model = nn.Sequential(
    nn.Linear(2, 2),    # 입력층 → 은닉층 : W(0), b(0)
    nn.Sigmoid(),       # 은닉층 활성화 함수 (비선형성!)
    nn.Linear(2, 1),    # 은닉층 → 출력층 : 로짓 z
)
criterion = nn.BCEWithLogitsLoss()          # 시그모이드 + BCE
optimizer = torch.optim.SGD(model.parameters(), lr=0.5, momentum=0.9)

for epoch in range(3000):
    loss = criterion(model(X), Y)   # 1) 순전파 2) 손실
    optimizer.zero_grad()           # 3) 기울기 초기화
    loss.backward()                 # 4) 오차 역전파 (autograd)
    optimizer.step()                # 5) 가중치 갱신
print(torch.sigmoid(model(X)).round())   # → [0, 1, 1, 0]
```

§1 논리회로 XOR (NumPy)

NAND·OR·AND 퍼셉트론을 손으로 연결

§2 순전파·역전파 손 구현 + autograd 검증

손으로 구한 dW 와 loss.backward() 결과가 1e-8 이내 일치

§3 초기화·활성화·최적화 비교

10개 시드 × 5개 설정의 성공률 — Xavier/He, GELU, AdamW 의 효과를 직접 확인 (슬라이드 21 그래프)

핵심 요약

6-1 다층 퍼셉트론의 구조와 개념

퍼셉트론

가중합 $w^T x$ 에 활성화 함수 f 를 적용. 결정 경계는 초평면 하나 → AND·OR·NAND 가능, XOR 불가

다층 퍼셉트론

은닉층 = 특징 변환. $h^{(k)} = f(W^{(k-1)}h^{(k-1)} + b^{(k-1)})$ 반복. 완전연결·순전파

비선형성

활성화 함수가 없으면 깊이는 무의미. 은닉층은 ReLU/GELU/SiLU, 출력층은 문제 유형에 따라

출력층

회귀 → 항등, 이진 → 시그모이드, 다중 → 소프트맥스 (PyTorch 에서는 손실 함수와 결합)

깊이와 이론

UAT: 표현 가능성 보장, 깊이 분리: 깊은 망의 효율. 초기화·정규화·잔차 연결이 깊은 학습을 가능케 함

최신 동향

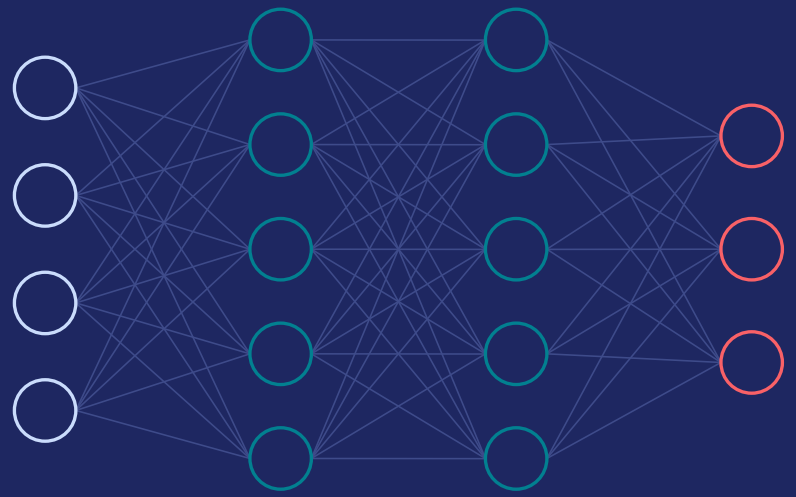
트랜스포머 FFN(SwiGLU), MoE, MLP-Mixer, KAN — 모두 «선형 + 비선형 블록»의 변주

질문 및 정리

Q & A

오늘의 핵심 정리

- XOR 은 «직선 하나»의 한계, 은닉층은 «특징 변환»
- MLP 순전파 = 행렬곱 + 바이어스 + 비선형 활성화의 반복
- 활성화 함수: 시그모이드(포화) → ReLU → GELU/SiLU/SwiGLU
- 소프트맥스 출력과 원-핫 정답 → 다음 강의: 손실 함수와 역전파
- 현대 블록: 초기화 + 정규화 + 잔차 + 드롭아웃
- FFN · MoE · MLP-Mixer · KAN 은 모두 MLP 의 변주



성명 : 최승호 · 이메일 : jcn99250@naver.com